10. Gyakorlat

Python kurzus

Big Data

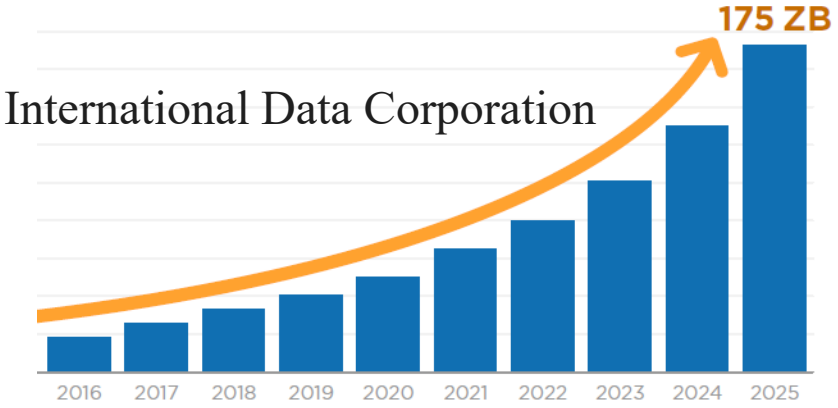
Bevezetés: Big Data alapfogalmak

1. Az adatok jelentősége, értéke: adatelemzés, adatvezérelt döntéshozatal

2. Az adat típusai:

- **Strukturálatlan adatok:** szöveges dokumentumok, e-mailek, képek és videók
- **Strukturált adatok:** táblázatok, adatbázisok

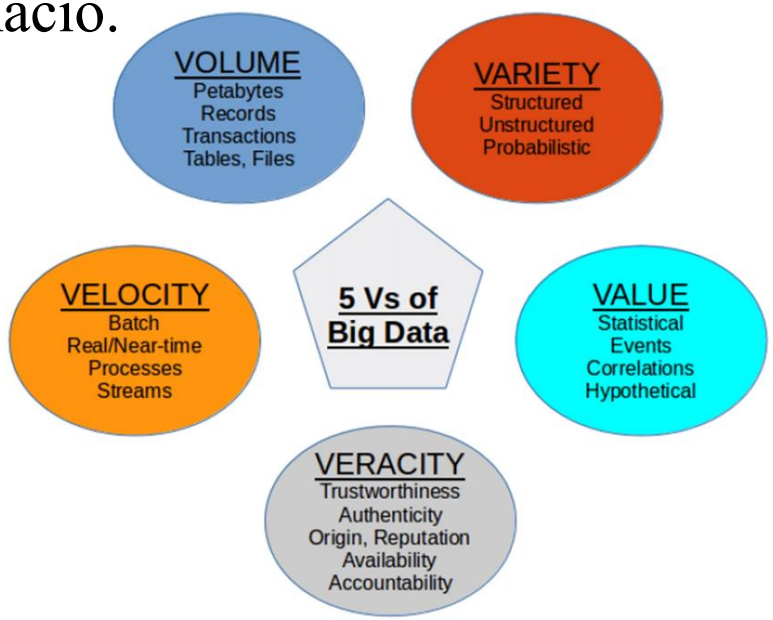
3. Adatok exponenciális növekedése:



4. Big Data definíció: Olyan adathalmazok, amelyek túl nagyok vagy összetettek ahhoz, hogy a hagyományos adatfeldolgozási módszerekkel kezelhetőek legyenek.
A Big Data rendszerek lehetővé teszik a nagy mennyiségű, gyorsan áramló és különböző forrásokból származó adatok valós idejű elemzését.

5. Big Data jellemzői: (5V modell):

- **Volume (Mennyiség)**: óriási adatmennyiségek.
- **Velocity (Sebesség)**: az adatok gyors beérkezése és feldolgozása.
- **Variety (Változatosság)**: különböző típusú és formátumú adatok.
- **Veracity (Hitelesség)**: az adatok minősége és pontossága.
- **Value (Érték)**: az adatokból kinyert hasznos információ.



https://www.researchgate.net/figure/The-five-dimensions-of-Big-Data-6_fig2_358990445

6. Hozzáférés szerinti csoportosítás:

- **Open Data**: Nyílt adatok, amelyek mindenki számára hozzáférhetők és szabadon használhatók, pl. kormányzati statisztikák, közlekedési adatok.
- **Private Data**: Privát adatok, amelyek egy szervezet vagy egyén tulajdonában vannak, és korlátozott hozzáférésűek, pl. ügyféladatbázisok, pénzügyi jelentések

7. Adatkezelési eszközök:

Hagyományos adatok esetén:

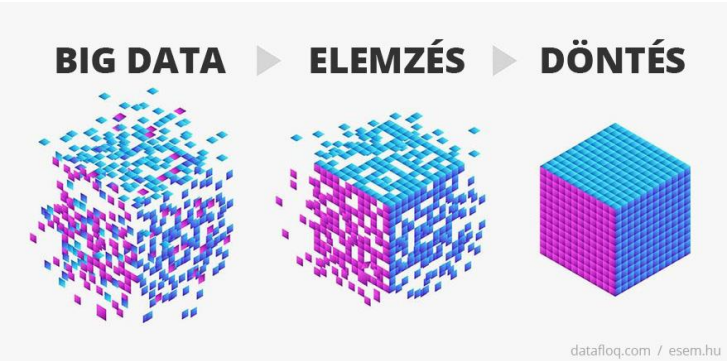
A relációs adatbázis-kezelő rendszerek (RDBMS), mint az Oracle, MySQL, SQL Server (kis- és közepes méretű adatok kezelése).

Big Data adatkezelési technológiára példák:

Strukturálatlan adatok: A Hadoop és a NoSQL adatbázisok (pl. MongoDB, Cassandra).
Strukturált adatok: SQLite, beágyazott adatbázis-kezelő rendszer.

8. Az adatelemzés lépései:

1. Adatok gyűjtése.
2. Adatok előkészítése és tisztítása.
3. Adatok vizualizációja.
4. Adatok elemzése.
5. Értelmezés és következtetések levonása.
6. Döntéshozatal és intézkedések végrehajtása.



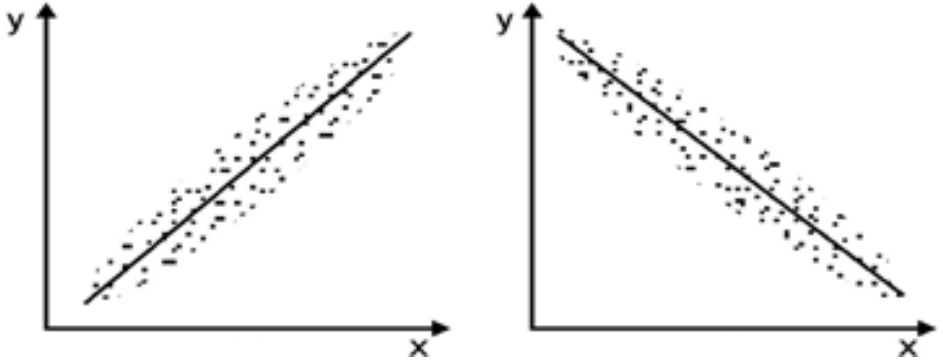
9. Adatelemzési módszerek

Descriptive (leíró): az adatok összegzése, a meglévő információk bemutatása:

- Statisztikai elemzések (átlag, medián, szórás).
- Adatvizualizációs technikák, pl. grafikonok, diagramok.
- Adatbányászati eszközök, az adatok közti összefüggések kimutatására.

Prediktív (előrejelző): a jövőbeli események előrejelzése:

- Regressziós elemzések, klasszifikáció.
- Gépi tanulási modellek, döntési fák, neurális hálózatok.



http://www.jgypk.hu/tamop15e/tananyag_html/spss/alapfogalmak3.html

https://developers.google.com/machine-learning/clustering/overview

Python eszközök a Big Data adatfeldolgozáshoz

NumPy könyvtár – Numerikus számításokhoz

- Tömbök használata (egy-, két- vagy többdimenziós).
- Hatékony matematikai műveletek tömbökkel.
- Beépített függvények pl. statisztikai számításokra.

NumPy példa:

```
import numpy as np
# Egydimenziós tömb létrehozása
arr = np.array([1, 2, 3, 4, 5])
print("Egydimenziós tömb:", arr)
# Többdimenziós (mátrix) tömb
matrix = np.array([[1, 2, 3], [4, 5, 6]])
print("Mátrix:\n", matrix)
```

```
# Alapvető műveletek: összeg, átlag
arr_sum = np.sum(arr)
arr_mean = np.mean(arr)
print("Összeg:", arr_sum)
print("Átlag:", arr_mean)
```

Pandas könyvtár – adatelemzéshez

Series: egydimenziós indexelt adattömb

```
import pandas as pd
series = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
print("Series példa:\n", series)
```

```
Series példa:
a    10
b    20
c    30
d    40
dtype: int64
```

DataFrame: kétdimenziós adattömb

```
data = {
   'Nev': ['Anna', 'Béla', 'Cecília'],
   'Kor': [29, 34, 22],
   'Varos': ['Budapest', 'Debrecen', 'Szeged']
}
df = pd.DataFrame(data)
print("\nDataFrame példa:\n", df)
# Adatokhoz való hozzáférés
print("\nElső sor adatai:")
print(df.iloc[0])
print("\nKor oszlop adatai:")
print(df['Kor'])
#print(df[['Kor','Nev']])
```

```
DataFrame példa:
        Nev  Kor       Varos
0      Anna   29   Budapest
1      Béla   34   Debrecen
2  Cecília    22      Szeged

Első sor adatai:
Nev              Anna
Kor                29
Varos     Budapest
Name: 0, dtype: object

Kor oszlop adatai:
0   29
1   34
2   22
Name: Kor, dtype: int64
```

Alapvető műveletek DataFrame-ekkel:

- **Adatok importálása és exportálása:**
  - CSV beolvasás: **pd.read_csv**('fajl_neve.csv')
  - CSV exportálás: **df.to_csv**('output.csv', index=False)
- **Alapvető adatvizsgálat:**
  - Az első néhány sor megtekintése: **df.head()**
  - Adatstatisztika: **df.describe()**
  - Adattípusok ellenőrzése: **df.dtypes**
- **Adatok manipulálása:**
  - Sorok és oszlopok kiválasztása: df[['Nev', 'Kor']]
  - Adatok szűrése: df[df['Kor'] > 30]
  - Oszlopok hozzáadása vagy módosítása: df['Uj_Oszlop'] = df['Kor'] * 2

Pandas példa: Adatok beolvasása és fájlba írása

```
df = pd.read_csv('fajl_neve.csv’)
print(df.head())                # DataFrame első sorai
#print(df.tail())              # DataFrame utolsó sorai
print(df.describe())            # Statisztikai jellemzők
print(df.dtypes)                #Oszlopok típusai
print(df[df['Kor'] > 30])       #Szűrés 30 év felettiek
#Új oszlop hozzáadása
df['Kor x2'] = df['Kor'] * 2
# Új dataframe szűréssel
debrecen_df = df[df['Varos'] == 'Debrecen']
# Átlagéletkor kiszámítása
atlag_eletkor = df['Kor'].mean()
print("\nÁtlagéletkor:", atlag_eletkor)
df.to_csv('kimenet.csv', index=False)
```

Az index=False eredménye, hogy a kimenet.csv fájlban nem lesz benne az első index oszlop.

```
head()
       Nev  Kor       Varos
0     Anna   29   Budapest
1     Béla   34   Debrecen
2 Cecília    22     Szeged
3     Anna   29   Budapest

describe()
                Kor
count      4.000000
mean      28.500000
std        4.932883
min       22.000000
25%       27.250000
50%       29.000000
75%       30.250000
max       34.000000

dtypes()
Nev     object
Kor     int64
Varos   object
dtype: object

 Nev  Kor    Varos
1   Béla   34  Debrecen

Átlagéletkor: 28.5
```

További DataFrame műveletek

Új oszlop hozzáadása:

```
df['jegyek'] = [4.5, 3.8, None, 4.5]
print("\nÚj oszloppal bővített DataFrame:\n", df)
```

Adatok rendezése (kor szerint csökkenően):

```
rendezett_df = df.sort_values(by='Kor', ascending=False)
print("\nRendezett DataFrame:\n", rendezett_df)
```

Átlag, maximum, szórás, módusz:

```
jegy_atlag = df['jegyek'].mean()
print("\nJegyek átlaga:", jegy_atlag)
max_kor = df['Kor'].max()
print("Legidősebb életkor:", max_kor)
szoras = df['jegyek'].std()
print("Jegyek szórása:", szoras)
modusz = df['jegyek'].mode()[0]
print("Jegyek módusza:", modusz)
```

Új oszloppal bővített DataFrame:

	Nev	Kor	Varos	Kor x2	jegyek
0	Anna	29	Budapest	58	4.5
1	Béla	34	Debrecen	68	3.8
2	Cecília	22	Szeged	44	NaN
3	Anna	29	Budapest	58	4.5

Rendezett DataFrame:

	Nev	Kor	Varos	Kor x2	jegyek
1	Béla	34	Debrecen	68	3.8
0	Anna	29	Budapest	58	4.5
3	Anna	29	Budapest	58	4.5
2	Cecília	22	Szeged	44	NaN

Jegyek átlaga: 4.266666666666667
Legidősebb életkor: 34
Jegyek szórása: 0.4041451884327381
Jegyek módusza: 4.5

Jupyter Notebook - Colab

gyak_10_colab.ipynb

Fájl Szerkesztés Nézet Beszúrás Futtatókörnyezet Eszközök Súgó

Parancsok + Kód + Szöveg

Fájlok

Elemezze a fájljait a Gemini által írt kóddal Feltöltés

..

sample_data

fajl_neve.csv

```
import pandas as pd
```

```
[ ] df=pd.read_csv("fajl_neve.csv")
    df.head()
```

```
[ ] df.describe()
```

```
[ ] df.dtypes
```

Az adatfeldolgozás lépései a Pandas könyvtár eszközeivel

1. **Adatok beolvasása**
2. **Adatok előkészítése és tisztítása**
3. **Szűrés, rendezés, aggregálás**
4. **Adattranszformációk, normalizálás, feltételes adatmanipuláció**
5. **Adatvizualizáció**

1. Adatok beolvasása

A leggyakoribb adatforrások közé tartoznak a **CSV**, **Excel** és **SQL** adatbázisok. Ezek beolvasására a Pandas különböző függvényeket biztosít.

CSV fájl beolvasása:

```
import pandas as pd

adatok_df = pd.read_csv('pelda_adatok.csv')
print("Beolvasott adatok:\n", adatok_df.head())
```

2. Adatok előkészítése és tisztítása

Hiányzó adatok eltávolítása és hiányzó adatok kitöltése (dropna, fillna):

```
# Sorok eltávolítása, ahol hiányzó adat van
adatok_df = adatok_df.dropna()
print("Hiányzó adatok eltávolítása után:\n", adatok_df.head())

# Hiányzó értékek kitöltése egy adott értékkel
adatok_df['kor'] = adatok_df['kor'].fillna(30)

# Hiányzó értékek kitöltése az oszlop átlagával
adatok_df['jegyek'] = adatok_df['jegyek'].fillna(adatok_df['jegyek'].mean())
print("Hiányzó értékek kitöltése átlaggal:\n", adatok_df.head())
```

Típuskonverzió egész számokká (astype(int)), dátumformátummá (to_datetime):

```
# 'kor' oszlop konvertálása egész számokká
adatok_df['kor'] = adatok_df['kor'].astype(int)
print("Adatok típusa konvertálás után:\n", adatok_df.dtypes)

# 'datum' oszlop konvertálása dátumformátummá
adatok_df['datum'] = pd.to_datetime(adatok_df['datum'])
```

Duplikált sorok eltávolítása (drop_duplicates):

```
adatok_df = adatok_df.drop_duplicates()
```

3. Szűrés, rendezés, aggregálás

Az adatmanipuláció kulcsfontosságú az adatelőkészítésében és az elemzési folyamatban.

Szűrés: Az adatok szűrése lehetővé teszi, hogy csak a megadott feltételnek megfelelő sorokat tartsuk meg egy új DataFrame-ben (**uj_df = df[df['kor'] > 25]**).

Rendezés: A rendezés lehetővé teszi az adatok sorainak sorba rendezését egy vagy több oszlop alapján (**sort_values(by='Kor', ascending=False)**).

Aggregálás: az adatok **összesítése, csoportosítása** (**groupby().size()**) és különböző metrikák alkalmazása, pl. **átlag, összeg, minimum, maximum**.

Példa

```
import pandas as pd
# Minta DataFrame létrehozása
adatok = {
   'nev': ['Anna', 'Béla', 'Cecília', 'Dávid', 'Emma'],
   'kor': [29, 34, 22, 30, 25],
   'varos': ['Budapest', 'Debrecen', 'Szeged', 'Pécs', 'Miskolc']
}
diakok_df = pd.DataFrame(adatok)
print(diakok_df)
```

	nev	kor	varos
0	Anna	29	Budapest
1	Béla	34	Debrecen
2	Cecília	22	Szeged
3	Dávid	30	Pécs
4	Emma	25	Miskolc

```
# Adatok szűrése, ahol a 'kor' nagyobb 25-nél, majd rendezés név szerint
szurt_rendezett_diakok = diakok_df[diakok_df['kor'] > 25].sort_values(by='nev')
print("Szűrt és név szerint rendezett diákok:\n", szurt_rendezett_diakok)

# Csoportosítás város szerint, majd a diákok számának megszámolása
varosi_diak_szam = diakok_df.groupby('varos').size()
print("Diákok száma városonként:\n", varosi_diak_szam)
```

Szűrt és név szerint rendezett diákok:

	nev	kor	varos
0	Anna	29	Budapest
1	Béla	34	Debrecen
3	Dávid	30	Pécs

Diákok száma városonként:

varos	
Budapest	1
Debrecen	1
Miskolc	1
Pécs	1
Szeged	1

4. Adattranszformációs és -manipulációs lehetőségek

Az adatok manipulációjával az a célunk, hogy **információkat nyerjünk ki** belőlük.

4.1. Adattranszformációk: oszlopok értékeinek módosítása, új oszlopok hozzáadása.

Példa: új oszlop létrehozása:

```
import pandas as pd
# Minta DataFrame létrehozása
adatok = {
   'nev': ['Anna', 'Béla', 'Cecília', 'Dávid', 'Emma'],
   'kor': [29, 34, 22, 30, 25],
   'varos': ['Budapest', 'Debrecen', 'Szeged', 'Pécs', 'Miskolc']
}
diakok_df = pd.DataFrame(adatok)

# Új oszlop hozzáadása, amely a kor kétszerese
diakok_df['kor_ketszer'] = diakok_df['kor'] * 2
print("Új oszloppal bővített DataFrame:\n", diakok_df)
```

	nev	kor	varos	kor_ketszer
0	Anna	29	Budapest	58
1	Béla	34	Debrecen	68
2	Cecília	22	Szeged	44
3	Dávid	30	Pécs	60
4	Emma	25	Miskolc	50

4.2. Adatok normalizálása

Az adatok normalizálása során az értékeket egy közös skálára helyezzük, ami segíthet az adatok összehasonlíthatóságában.

Értékek skálázása 0 és 1 közé:

```
diakok_df['kor_normalizalt'] = (diakok_df['kor'] - diakok_df['kor'].min()) /
(diakok_df['kor'].max() - diakok_df['kor'].min())

print("Normalizált 'kor' oszlop:\n", diakok_df)
```

Normalizált 'kor' oszlop :

	nev	kor	varos	kor_ketszer	kor_normalizalt
0	Anna	29	Budapest	58	0.636364
1	Béla	34	Debrecen	68	1.000000
2	Cecília	22	Szeged	44	0.000000
3	Dávid	30	Pécs	60	0.727273
4	Emma	25	Miskolc	50	0.272727

4.3. Feltételes adatmanipuláció

Az adatok manipulációja során szükséges lehet egyes sorokat vagy oszlopokat feltételek alapján módosítani.

Új oszlop létrehozása feltétel vizsgálattal (apply()):

```
# Új oszlop, amely jelöli, ha a kor 30 év felett van (Igen/Nem)
diakok_df['30_feletti'] = diakok_df['kor'].apply(lambda x: 'Igen' if x > 30 else 'Nem')
print("Feltételes oszloppal bővített DataFrame:\n", diakok_df)
```

Feltételes oszloppal bővített DataFrame:

	nev	kor	varos	kor_ketszer	kor_normalizalt	30_feletti
0	Anna	29	Budapest	58	0.636364	Nem
1	Béla	34	Debrecen	68	1.000000	Igen
2	Cecília	22	Szeged	44	0.000000	Nem
3	Dávid	30	Pécs	60	0.727273	Nem
4	Emma	25	Miskolc	50	0.272727	Nem

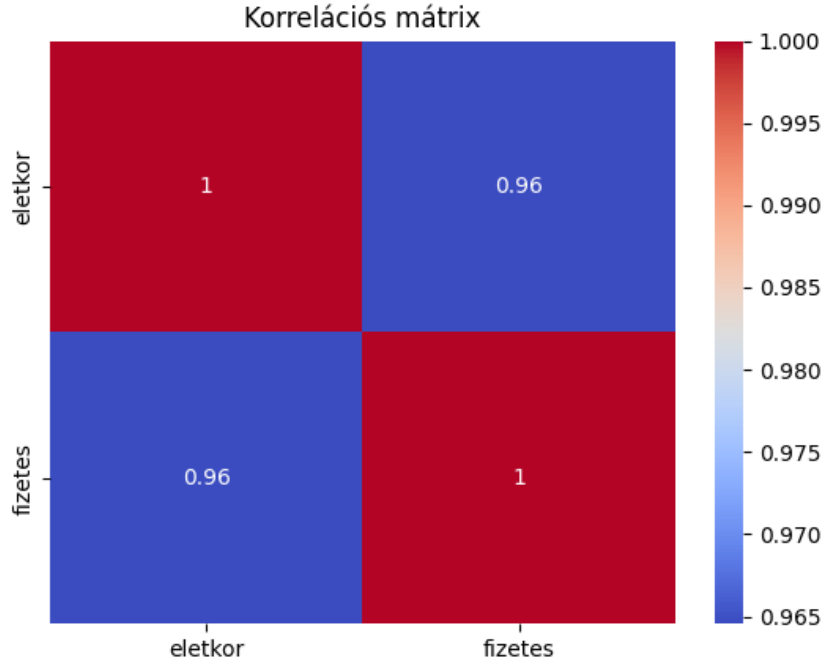
Adatvizualizáció 1. példa

Korreláció vizsgálata (corr(), heatmap())

```
import matplotlib.pyplot as plt
import seaborn as sns

adatok = {
   'eletkor': [23, 45, 34, 25, 32, 40, 29, 48, 37, 22],
   'fizetes': [250, 500, 300, 260, 320, 480, 290, 520, 410, 240]
}

adat_df = pd.DataFrame(adatok)

# Korrelációs mátrix kiszámítása
korrelacio = adat_df.corr(method='pearson')
print("\nKorrelációs mátrix:")
print(korrelacio)

# Korrelációs mátrix megjelenítése
sns.heatmap(korrelacio, annot=True, cmap='coolwarm')
plt.title('Korrelációs mátrix')
plt.show()
```

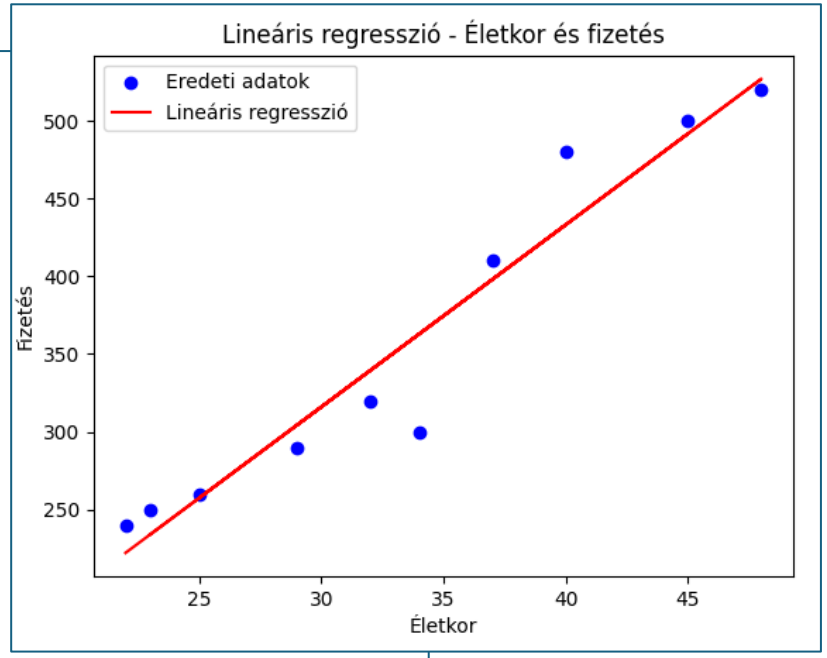
Korrelációs mátrix:

	eletkor	fizetes
eletkor	1.000000	0.964549
fizetes	0.964549	1.000000



Adatvizualizáció 2. példa
Lineáris regresszió vizsgálata

```
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Független (x) és függő (y) változók kiválasztása
x = adat_df[['eletkor']]
y = adat_df['fizetes']

modell = LinearRegression()    # Lin. regr. modell létrehozása
modell.fit(x, y)

# Egyenes egyenletének kiírása

print(f'Y = {modell.coef_[0]:.2f} * X + {modell.intercept_:.2f}’)

y_elorejelzes = modell.predict(x)      # Előrejelzés az adatokra

# Diagram rajzolása
plt.scatter(adat_df['eletkor'], adat_df['fizetes'], color='blue', label='Eredeti adatok')
plt.plot(adat_df['eletkor'], y_elorejelzes, color='red', label='Lineáris regresszió')
plt.xlabel('Életkor')
plt.ylabel('Fizetés')
plt.title('Lineáris regresszió - Életkor és fizetés')
plt.legend()
plt.show()
```



Y = 11.73 * X + -35.92

Köszönöm a figyelmet!

Konzultáció

Minden héten csütörtökön 18:00 – 20:00